Make a plan for the following project with hugging face api and pyhton and uv.Use lightweight package for UI:

Interactive Kids' Storyteller with Whisper-Small: An educational app where a child reads a storybook out loud. The app uses Whisper to listen to the child's voice. If they pronounce a word correctly, the digital book triggers animations, sound effects, or advances to the next page. If they stumble, the app gently highlights the word and pronounces it for them.

Here's a practical build plan for an **Interactive Kids' Storyteller** using the Hugging Face API, Python, `uv`, and **Gradio** as the lightweight UI. Gradio supports microphone input directly, which makes it well suited for a read-aloud app.[1] [2]

Architecture

```
Child reads page aloud
        ↓
Gradio microphone capture
        ↓
Python audio preprocessing
        ↓
Hugging Face Inference API: openai/whisper-small
        ↓
Word-alignment / pronunciation comparison
        ↓
UI feedback: animation, sound, next page, or gentle correction
```

Use **Whisper-Small via Hugging Face's Inference API** rather than loading the model locally, so the app remains lightweight on the child's device. The Hugging Face `InferenceClient` supports automatic speech recognition with an audio file and a chosen model.[3] [4]

Tech stack

Layer	Choice	Why
Language/runtime	Python 3.11+	Strong audio/ML ecosystem
Package manager	`uv`	Fast dependency management and virtual environments
UI	Gradio	Lightweight browser UI with built-in microphone `Audio` component [1] [2]
Speech recognition	Hugging Face Inference API + `openai/whisper-small`	Avoids local GPU/model download; Whisper-Small is a reasonable balance of accuracy and size [5]

Layer	Choice	Why
Audio handling	`soundfile`, `numpy`, optionally `librosa`	Convert, trim, normalize, and validate recordings
Pronunciation feedback	Fuzzy matching + optional phonetic comparison	Tolerates child-like variations while catching real mispronunciations
Content	JSON storybook files	Easy for teachers/parents to add pages and target words

Project structure

```
kids-storyteller/
├── pyproject.toml
├── .env
├── app.py
├── src/
│   ├── __init__.py
│   ├── ui.py
│   ├── story_engine.py
│   ├── speech.py
│   ├── pronunciation.py
│   ├── feedback.py
│   └── assets.py
├── stories/
│   ├── the_little_dragon.json
│   └── space_cat.json
├── assets/
│   ├── images/
│   ├── sounds/
│   └── animations/
└── tests/
    ├── test_pronunciation.py
    └── test_story_engine.py
```

Setup with uv

Initialize the project:

```
uv init kids-storyteller
cd kids-storyteller
uv add gradio huggingface-hub python-dotenv soundfile numpy rapidfuzz
uv add --dev pytest
```

Create `.env`:

```
HF_TOKEN=your_huggingface_token
```

Load it in Python:

```
from dotenv import load_dotenv
load_dotenv()
```

The Hugging Face client expects an API key, typically supplied through an environment variable such as `HF_TOKEN`.[3]

Core dependencies

In `pyproject.toml`, the key runtime dependencies should include:

```
dependencies = [
    "gradio>=4.0",
    "huggingface-hub>=0.24",
    "python-dotenv>=1.0",
    "soundfile>=0.12",
    "numpy>=1.26",
    "rapidfuzz>=3.0"
]
```

`huggingface-hub` provides `InferenceClient`, whose `automatic_speech_recognition()` method returns transcribed text from an audio input.[4]

Storybook format

Store each story as JSON so pages, target words, rewards, and hints are configurable.

```
{
  "title": "The Little Dragon",
  "language": "en",
  "pages": [
    {
      "page_number": 1,
      "text": "Once upon a time, there was a little dragon named Pip.",
      "target_words": ["dragon", "Pip"],
      "image": "assets/images/page1.png",
      "success_sound": "assets/sounds/sparkle.wav",
      "success_animation": "assets/animations/dragon_flies.gif",
      "hint": "Dragon starts with the sound 'dr'."
    }
  ]
}
```

This keeps the app content-driven: adding a new story requires no code changes.

Speech recognition module

Create `src/speech.py`:

```
import os
from huggingface_hub import InferenceClient
```

```
client = InferenceClient(
    provider="hf-inference",
    api_key=os.environ["HF_TOKEN"],
)

def transcribe_audio(audio_path: str) -> str:
    result = client.automatic_speech_recognition(
        audio_path,
        model="openai/whisper-small",
    )
    return result.text.strip().lower()
```

Whisper-Small is available through Hugging Face and can be used for automatic speech recognition; using the hosted Inference API keeps the client-side app light. [5] [3]

Pronunciation logic

Avoid treating Whisper output as a perfect pronunciation score. Use a forgiving comparison strategy:

1. Normalize text: lowercase, remove punctuation, expand simple contractions.
2. Compare the child's transcript with the page's expected words.
3. Use fuzzy matching with `rapidfuzz` to allow small recognition errors.
4. Mark a word correct if its similarity is above a threshold, for example 80–85%.
5. If incorrect, highlight the word, show a friendly hint, and play an audio pronunciation.

Example:

```
from rapidfuzz import fuzz

def is_word_correct(expected: str, heard: str, threshold: int = 82) -> bool:
    expected = expected.lower().strip()
    heard = heard.lower().strip()
    return fuzz.ratio(expected, heard) >= threshold
```

For better feedback, compare each expected word against the closest word in the transcript rather than requiring an exact sentence match.

Gradio interface

Create `src/ui.py`:

```
import gradio as gr

from story_engine import StoryEngine
from speech import transcribe_audio
from pronunciation import evaluate_reading

engine = StoryEngine("stories/the_little_dragon.json")
```

```
def handle_reading(audio, page_index):
    if audio is None:
        return engine.render_page(page_index), "Please read the page aloud."

    audio_path = audio
    heard_text = transcribe_audio(audio_path)
    result = evaluate_reading(
        expected_words=engine.get_target_words(page_index),
        heard_text=heard_text,
    )

    if result["all_correct"]:
        engine.trigger_success(page_index)
        next_page = engine.next_page(page_index)
        return engine.render_page(next_page), "Great reading! "

    return engine.render_page(page_index, highlight=result["incorrect_words"]), resu

with gr.Blocks(title="Kids' Storyteller") as demo:
    page_display = gr.HTML()
    feedback = gr.Markdown()
    audio_input = gr.Audio(
        sources=["microphone"],
        type="filepath",
        label="Read this page aloud",
    )
    page_state = gr.State(0)

    audio_input.stop_recording(
        handle_reading,
        inputs=[audio_input, page_state],
        outputs=[page_display, feedback],
    )

demo.launch()
```

Gradio's `Audio` component supports `"microphone"` as a source, so children can read directly in the browser without a separate recording workflow. [2] [1]

Feedback design

Keep the experience encouraging rather than test-like.

Child behavior	App response
Reads target word correctly	Play sparkle/chime sound, animate character, optionally advance page
Reads most words correctly	Praise progress, gently repeat only the missed word
Mispronounces a word	Highlight it softly, show phonetic hint, play correct pronunciation
Says nothing / audio unclear	Ask child to try again; never display an error-style message
Repeated difficulty	Offer "Listen together" mode and reduce required words per page

Important: Whisper transcribes what it hears, but it does not reliably provide phoneme-level pronunciation scoring. For a first version, use word-level matching; later, add a dedicated pronunciation-assessment model or phoneme alignment if you need more precise feedback.

MVP milestones

Week 1: Foundation

- Initialize project with `uv`.
- Add Gradio microphone capture.
- Create one two-page JSON story.
- Send recorded audio to Hugging Face Whisper-Small.
- Display raw transcript in the UI.

Week 2: Reading feedback

- Implement text normalization.
- Add fuzzy word matching.
- Highlight correct and missed target words.
- Add success/failure sound effects.
- Add page advancement after successful reading.

Week 3: Kid-friendly polish

- Add illustrations, animations, and friendly mascot.
- Add “hear the word” pronunciation audio.
- Add progress stars and encouragement messages.
- Add parent/teacher settings: difficulty, target words, autoplay hints.
- Test with real children and adjust thresholds.

Week 4: Reliability and release

- Add retry logic and timeout handling for API calls.
- Cache stories and assets locally.
- Add basic privacy controls: do not retain audio unless a parent opts in.
- Add tests for pronunciation matching and story progression.
- Package the app with `uv run app.py` or deploy via Hugging Face Spaces.

Privacy and child safety

- Do not store children's audio by default.
- Show a clear parent-facing notice that speech is sent to a cloud speech service.
- Use only fictional names and avoid asking for personal information.
- Keep all correction messages positive; avoid scores like "failed."
- Add a "grown-up setup" screen for story selection, microphone permission, and data preferences.

First runnable goal

Your first milestone should be simple: **record one sentence, transcribe it with Whisper-Small, and highlight whether the word "dragon" was said.** Once that loop works, layer in animations, multi-page stories, and more sophisticated pronunciation feedback.

⁂

1. https://gradio.app/api/markdown/audio
2. https://gradio.app/4.44.1/docs/gradio/audio
3. https://huggingface.co/docs/inference-providers/en/providers/hf-inference
4. https://huggingface.co/docs/huggingface_hub/package_reference/inference_client
5. https://huggingface.co/openai/whisper-small
6. https://huggingface.co/TheStigh/nb-whisper-small-ct2?library=transformers
7. https://huggingface.co/api/resolve-cache/models/Kadamba/whisper-kannada-small/800932eada631c50d7ff24740d03af96d289cc9c/README.md?download=true&etag="c802483067434cb84775f377e2c2fd1923b56849"
8. https://huggingface.co/api/resolve-cache/datasets/John6666/forum3/155149c758a66ea955d57f160037da73b8492306/ai_transcription_learn_1.md?download=true&etag="dab8aba9c2054fc7c0509b349d5f8275f1ee1863"
9. https://github.com/imtisaalm/funasr/blob/main/examples/openai_api/GRADIO.md
10. https://github.com/Magenta91/speech-eval-pipeline
11. https://huggingface.co/datasets/John6666/forum3/blame/f32fecb18793975ac19f840ff55ab6f0a15cdbbb/ai_transcription_learn_1.md
12. http://gradio.app/api/markdown/guide/getting-started-with-the-python-client
13. https://www.atlantic.net/gpu-server-hosting/audio-transcription-effortlessly-with-distill-whisper-ai-on-ubuntu-24-04-gpu-server/
14. https://huggingface.co/api/resolve-cache/models/KBLab/kb-whisper-small/3564d61a42fc210ceaa55a22a96dd64478959c78/README.md?download=true&etag="33e0060b4350019c67dad7af119bf48859627320"
15. https://huggingface.co/docs/inference-providers/providers/hf-inference